

Mandatory Assignment 2 - Random Numbers



LEARNING OBJECTIVES

- Reading input and validating it with `try/except.`
- Working with Python lists (creation, traversal, copying).
- Implementing your own bubble sort (no built-ins)



The most typical mistakes that students do:

- They do not read the instructions carefully. Please see how you will be graded section
 - especially how the sorted is printed based on even numbers and odd numbers
- they do not use constants
- they do not read the coding rules
- they do the assignment without reading about the theory
- they have not seen the assignment overview video on Canvas



Prerequisites

The assignment requires you to study the material up to first part of Module 4 and bring your knowledge and feedback from previous assignments.

From Module 4, you should have completed

- **[4.3 Error handling in Python Programming](https://canvas.ltu.se/courses/24507/pages/4-dot-3-error-handling-in-python-programming)** (<https://canvas.ltu.se/courses/24507/pages/4-dot-3-error-handling-in-python-programming>).
- **[4.4 Data Structures in computer programming](https://canvas.ltu.se/courses/24507/pages/4-dot-4-data-structures-in-computer-programming)** (<https://canvas.ltu.se/courses/24507/pages/4-dot-4-data-structures-in-computer-programming>).
- **[4.5 Lists in Python programmin](https://canvas.ltu.se/courses/24507/pages/4-dot-5-lists-in-python-programming)** (<https://canvas.ltu.se/courses/24507/pages/4-dot-5-lists-in-python-programming>).
- **[4.6 Data structures as function parameters in Python programming](https://canvas.ltu.se/courses/24507/pages/4-dot-6-data-structures-as-function-parameters-in-python-programming)** (<https://canvas.ltu.se/courses/24507/pages/4-dot-6-data-structures-as-function-parameters-in-python-programming>).
- **[Quiz 4A - Error handling in Python programming](https://canvas.ltu.se/courses/24507/assignments/204125)** (<https://canvas.ltu.se/courses/24507/assignments/204125>).

- [Quiz 4B - Lists in Python Programming](https://canvas.ltu.se/courses/24507/assignments/204126)
(<https://canvas.ltu.se/courses/24507/assignments/204126>)
- [Practice Assignment 4A - Error Handled Calculator](https://canvas.ltu.se/courses/24507/assignments/204105)
(<https://canvas.ltu.se/courses/24507/assignments/204105>)



Task

Write a program that works with random numbers.

1. Ask the user how many random numbers to generate (an integer ≥ 1).
2. Generate that many integers in the range **0–999** (inclusive) and store them in a list of exactly that size.
3. The program must not crash regardless of input:
 - Handle invalid input with `try/except` and print exactly `INVALID_INPUT_MESSAGE`.
 - If the amount is too large for the system, catch `MemoryError` and print: *Impossible to handle the requested amount of numbers on this system*.
 - Handle any unexpected `IndexError` during list operations with a clear message.
 - Depending upon how you write the code, handling of `IndexError` may not be needed. Add a try-except block for `IndexError` only if your code could potentially produce it. For example, use of a insert method.
4. Print the numbers in the order generated (space-separated).
5. Split the numbers into **evens** and **odds** (you may use separate lists).
6. Sort:
 - Evens in **increasing** order using your bubble sort.
 - Odds in **decreasing** order using your bubble sort (or ascend then reverse with a loop).
7. Print the arranged sequence as: `even... - odd...`
 - If there are no evens, print *No Even Numbers* on the left side.
 - If there are no odds, print *No Odd Numbers* on the right side.
8. Print the counts of even and odd numbers exactly as in the examples.

Notes

- Use only features covered so far: lists, loops (for/while), if, simple functions, try/except.
- Do not use external libraries beyond Python's standard random module.
- Do not use list methods or constructs we haven't covered for sorting (no sort, sorted, or clever one-liners).



Example run of the program

Below, you can see what a run of the program should look like:

How many random numbers in the range 0 - 999 are desired? 11

Here are the random numbers:

326 300 309 791 260 341 500 2 495 548 960

Here are the random numbers arranged:

2 260 300 326 500 548 960 - 791 495 341 309

Of the above 11 numbers, 7 were even and 4 odd

In case there are only odd or even numbers then output could look like below

How many random numbers in the range 0 - 999 are desired? 5

Here are the random numbers:

326 300 260 500 2

Here are the random numbers arranged:

2 260 300 326 500 - No Odd Numbers

Of the above 5 numbers, 5 were even and 0 odd

How many random numbers in the range 0 - 999 are desired? 4

Here are the random numbers:

309 791 341 495

Here are the random numbers arranged:

No Even Numbers - 791 495 341 309

Of the above 4 numbers, 0 were even and 4 odd



Tips

- Start with the template available on Codegrade
- Even check: $n \% 2 == 0$
- Use `random.randint(0, 999)` for each number.
- Bubble sort reminder (ascending):
 - Repeatedly traverse the list, swapping adjacent out-of-order elements.
 - Stop after you've made a full pass with no swaps.
- To get descending odds, either:
 - Bubble sort ascending and then reverse with a loop, or
 - Change the comparison so larger values "bubble" forward.



How will you be graded / What do you need to take care of

- You will be graded and checked for functional and non-functional requirements
- Functional requirements

- The user can enter any positive integer (theoretical upper bound is only limited by available memory). If the system cannot allocate, you catch the error and show a clear message
- All invalid inputs are handled with try/except and print exactly: `INVALID_INPUT_MESSAGE` (This constant is provided in the CodeGrade template. If you are running locally, define it at the top: `INVALID_INPUT_MESSAGE = "Invalid Input".`)
 - You need to handle `ValueError` and `MemoryError` in try-except blocks. Two separate try except blocks both these exceptions are needed, do not combine them
- Randomized numbers should be in the range of 0 - 999
- The printing of numbers should have the same format as the example run
 - First, unsorted numbers are printed
 - Even numbers in increasing order are printed, followed by a '-' sign
 - Odd numbers in decreasing order are printed.
 - The count of even and odd numbers is printed.
- Non-functional requirements
 - Strictly follow the coding rules
 - Use only the constructs and knowledge you have learnt in this course.

Codegrade Specific Issues - most common

- I have used try-except blocks for both `MemoryError` and `ValueError`, but still see codegrade says wrong
 - Please see instructions above, I clearly say "You need to handle `ValueError` and `MemoryError` in try-except blocks. Two separate try except blocks, both these exceptions are needed, do not combine them."
 - What does that mean?
 - It means only one except block per try block, in other words, I try....except block for each error.
- Data Correctness tests fail
 - There are tests that test the output is exactly as it is supposed to be. The most common mistakes when this fails is
 - The sorting order for even and odd numbers is wrong. Please note, one is ascending and one is descending.
 - There is no summary at the end like in the example above.
 - There are extra print statements
 - You print the whole list in a print
 - for example, if there is a list called `random_numbers`, you cannot just do `print(random_numbers)`, you need to print them individually using space

Stuck? Need a Guide?

You should try this first by yourself. If you are stuck or need more help. Then use the [Guide for Mandatory Assignment 2 - Random Numbers](https://canvas.ltu.se/courses/24507/pages/guide-mandatory-assignment-2-random-numbers) ([https://canvas.ltu.se/courses/24507/pages/guide-](https://canvas.ltu.se/courses/24507/pages/guide-mandatory-assignment-2-random-numbers)